The Ghost v0.02: From Declared Erasure to Measured Disappearance

Crypto-shredding, scrypt audit, time window, and GSM8K evidence

dr. Francesco Bulla — Bologna, 10/10/2026

Local preprint, unpublished draft — v0.02

Abstract

In v0.01 the Ghost "disappeared" by declaration: emptying two Python dicts. This note turns erasure into a measured property. We define three levels of disappearance (L1 application stores, L2 process memory, L3 third parties and provider), replace obfuscation with stdlib crypto-shredding, the HMAC commitment with memory-hard scrypt, the 13-digit code with a canonical 19-digit code, and add a declared time window that resolves the verifiable-versus-secret paradox instead of dodging it. Every guarantee is conditioned on its own measurement report: 44 green tests, filesystem scan, mandatory positive and negative controls, Linux measurement via Docker. On 50 GSM8K problems with DeepSeek-V4.1-Flash the Ghost scores 47/50 against 46/50 for both direct and hidden-CoT baselines, with verified audit and complete amnesia on every cycle. The signal is weak and we say so: this is a pilot, not a proof. The ground for v0.03 is ready.

1 What the Ghost is and why it differs

The Ghost is an internal reasoning layer that emerges before generation, produces a thought in a private numeric code, has it re-read by the same model, reformulates the request, then destroys the key that makes the thought readable. The user sees only the final output; the audit sees only commitments, never contents.

Approach	Thought	Erasure	Who can re-read it
Visible CoT	in the text	none	anyone
Hidden scratchpad	hidden from user	declared emptying	operator, provider
Latent reasoning	continuous vectors	none	whoever holds the weights
Ghost v0.02	19-digit code	**key destruction**	nobody after T

The difference is not where the thought lives, but **how it dies**: not by forgetfulness, by destruction of the material that makes it readable. The rest — the reformulation — survives by choice: it is the Ghost's utility, not a leak.

2 The problem: disappearance was a sentence

Review measurements on the v0.01 prototype: the runtime-generated code leaves 3 copies in process memory, 4 when held in a single `bytearray`, 10 while building the prompt, 8 after wipe plus garbage collector. With the code written as a source literal, 31 copies: every f-string, log, traceback, HTTP request and socket buffer spawns one. And the v0.01 digit shift is a Caesar cipher with 10 possibilities: readable by hand. Hence the threat model, one page, three levels:

Level	Promise	v0.02 status
L1 — application	no reference in stores and files	guaranteed and measured
L2 — process	no unexpected copy in inspectable RAM	measured, never guaranteed
L3 — provider and network	no copy anywhere	never claimed

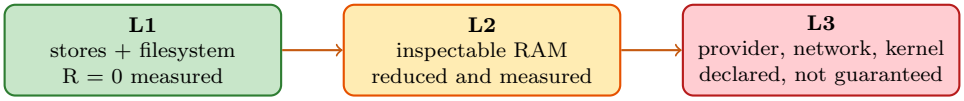


Figure 1: The three disappearance levels. Only L1 is a guarantee; L2 is a measurement; L3 is a declared limit.

On 43 bits — 13 digits — verifiable and non-invertible are mutually exclusive: if the commitment is verifiable, anyone can enumerate 10^{13} candidates and open it. v0.02 takes both exits: it raises entropy to 19 digits and declares window T.

3 v0.02 architecture: what actually changes

`src/fantasma`: `secret.py` (`SecretBytes` in wipeable bytearray, `OneTimeKey`, `SecretRegion` with construction-time wipe even on exception), `crypto.py` (stdlib encrypt-then-MAC over bytearray, scrypt, `TimedCommitment`), `ghost_window.py` (guard rail against accidental fork, traceback release), `hygiene.py` (punctual mlock on the region, never mlockall), `forensics.py` (runtime canaries, 7 encodings, store and filesystem scan, R as unexpected copies, conditional claim), `pipeline_v02.py` (same `.process()` interface, new interior), `bench_gsm8k.py` (GSM8K harness).

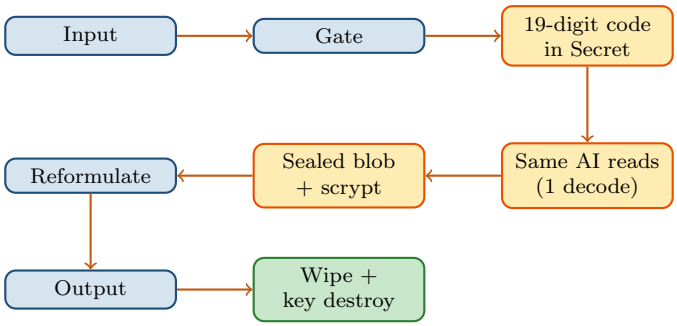


Figure 2: v0.02 cycle. The cache holds only undecipherable blobs; the key dies by memset.

Canonical 19-digit code (63.1 bits), never an integer. The leading zero is significant; numeric serializations are caught by the scanner's `int-decimal` and `int-hex` variants.

Measured scrypt, not hoped. N= 2^{14}, r= 8, p= 1: steady-state median ∼44 ms per guess on one core, ∼14,000 core-years over 10^{13}. But scrypt is bandwidth-bound: on 1 GPU ∼3.2 years for 13 digits, on 100 GPUs ∼12 days. At 19 digits: millions of years even on GPUs. The per-core number is not the defense; entropy is.

Declared time window. Within T the commitment verifies via a live-window keyed tag; after `expire()` only one-way scrypt remains. It is the Vision's "time limits" item applied to the audit: verifiable within T, one-way after T.

Wipe by construction. `SecretRegion` memsets via ctypes in `__exit__` always, even on early return or exception. Measured: a retained reference plus early return still yield a zeroed buffer. `expose_mutable()` remains a low-level API; the pipeline goes through the manager.

Punctual hygiene. No mlockall: only mlock on the secret region with a per-page report. The report says "designated region protected", never "nothing in swap": copies outside the region

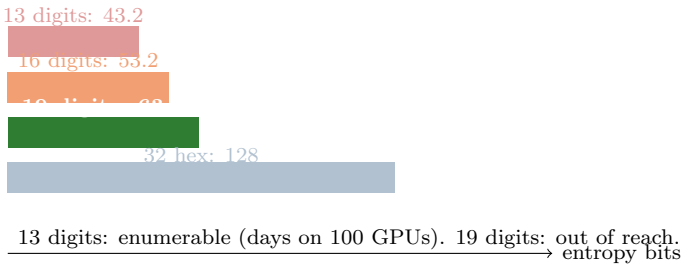


Figure 3: The real lever is entropy, not the KDF. At 43 bits, verifiable and secret exclude each other.

are not locked. The L2 claim requires a successful lock plus core dumps off plus a zero scan, otherwise it is not made.

4 Conditional claim: guarantee versus evidence

A guarantee is a sentence with a guard in front. `evaluate_claim()`: L1 always claimable on its own stores; L2 only with green hygiene and a clean scan; L3 never. R = found copies minus declared legitimate copies: the scanner holds the canary to search for it, so that copy is subtracted, otherwise R measures itself. Mandatory positive and negative controls on every run: without a positive that finds the injected leak, R=0 means nothing. "Not found" holds only for the seven searched encodings: raw, UTF-16LE/BE, hex, base64, int-decimal, int-hex.

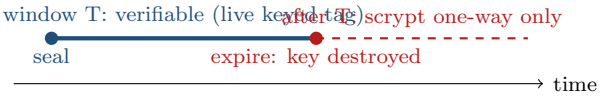


Figure 4: The paradox resolved instead of dodged: verifiable within T, one-way after T.

5 Measured limits, not hidden ones

`bytes` cannot be overwritten, `bytearray` can: the key lives in a mutable buffer, dropping the reference is hygiene, not erasure. `del` and `gc` free without overwriting: zero L1 holds for the stores, not for RAM. `f_locals` scrubbing is write-through only since Python 3.13 and only on live frames: on traceback-suspended frames the write does not propagate (measured, verified zero); the robust path stays `finally` plus traceback release. `ghost_window` prevents accidental fork from Python code, it is not a barrier against subprocess or C extensions; on Windows fork does not exist and the start method is spawn. AES-GCM via OpenSSL copies the plaintext in C: for a disappearance claim in-place stdlib is architecturally better, with AEAD migration still planned for v1.0. On Windows without `/proc` and without mlock, L2 is unmeasurable and unclaimed: that is the guard working, not a failure.

Docker measurement (Linux 6.6 WSL2, Python 3.11.17, green hygiene, lock on 19 bytes), 19-digit canary, memory regions containing the canary:

6 Evidence: GSM8K at 50

Harness in `src/fantasma/bench_gsm8k.py`: 50 test-split problems, fixed seed 20261010, three arms on the same DeepSeek-V4.1-Flash with the same final `#### number` format. Results in `outputs/benchmark/`.

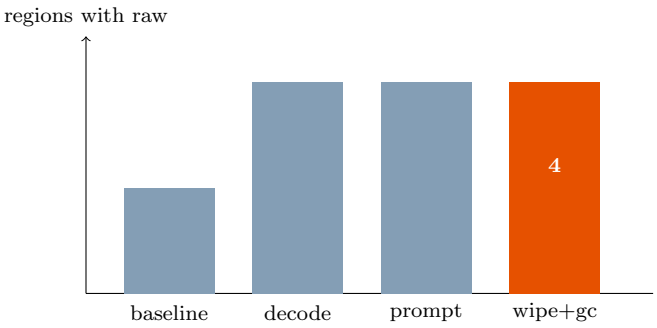


Figure 5: L2 does not return to zero: baseline 2, after decode 4, after wipe and gc still 4. Reduced and measured.

Arm	Accuracy	Mean time	Notes
Direct	46/50 = 0.920	2.6 s	one call
Hidden CoT	46/50 = 0.920	2.8 s	one call
Ghost v0.02	**47/50 = 0.940**	10.7 s	two calls + seal

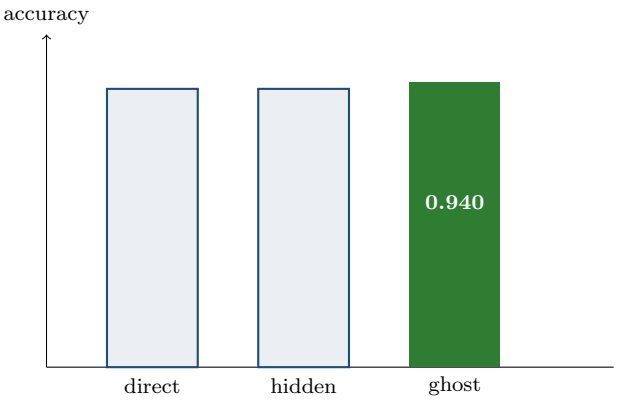


Figure 6: GSM8K at 50: one notch above, same model. A pilot, not a proof.

Guarantee side on the ghost cycle: ghost active 48/50 (2 legitimate standbys), audit verified 50/50, memory and cache empty 50/50, valid format throughout. A single problem separates the arms — gold 54, ghost right, the others wrong together at 27 — and there is no case where the ghost fails alone. With a model already at 92% the signal is weak by construction: we declare it a pilot.

7 What changes since v0.01

Caesar → stdlib shredding; naked HMAC → scrypt; 13 digits → canonical 19; reference drop → construction-time memset; mlockall → punctual mlock; "it disappears" → L1/L2/L3 with guards; windowless audit → window T; 12 tests → 44 tests; no benchmark → GSM8K at 50 with three arms.

8 Toward v0.03: the prepared ground

The question stands unanswered since the first note: does the Ghost actually improve answers over hidden CoT? The ground is ready: three-arm harness, fixed seeds, validated format. v0.03 scales to 200 problems with stratified sampling on cases where direct fails, adds statistical significance,

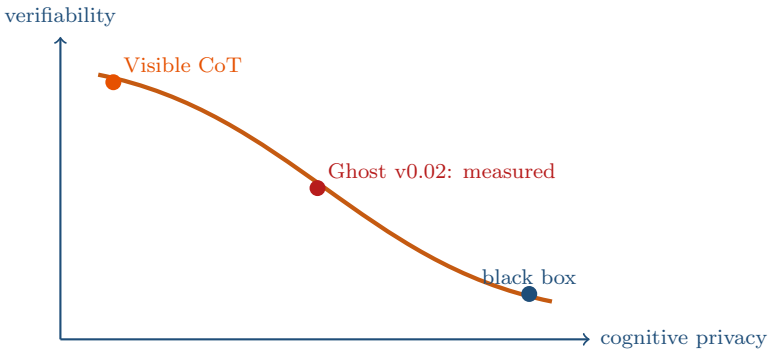


Figure 7: Same curve as v0.01, but the Ghost's point now has measured coordinates.

tries the local open-weights model where L3 narrows, and introduces independent gate supervision with selective reset. The rest — supervision, extended time limits, the local/remote split — is the v1.0 roadmap.

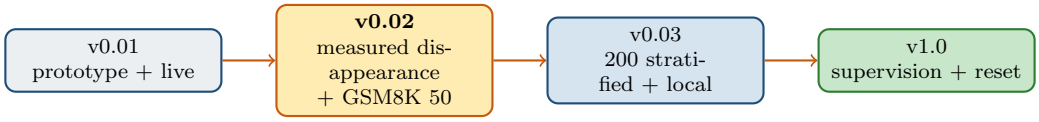


Figure 8: Updated roadmap: measurement exists, now significance.

9 Status and reproducibility

44 green tests (`pytest tests/`), 12 originals intact. L2 measurements on Linux 6.6 WSL2, Python 3.11.17, Docker with memlock unlocked; application measurements on Windows 10.0.26200, Python 3.13.13. Dataset `openai/gsm8k` test split, seed 20261010, model `deepseek-flash` with `reasoning_effort=high`. Key from env only, never in code. Otherwise it stays in the notebook: this one, for now, measures instead of promising.